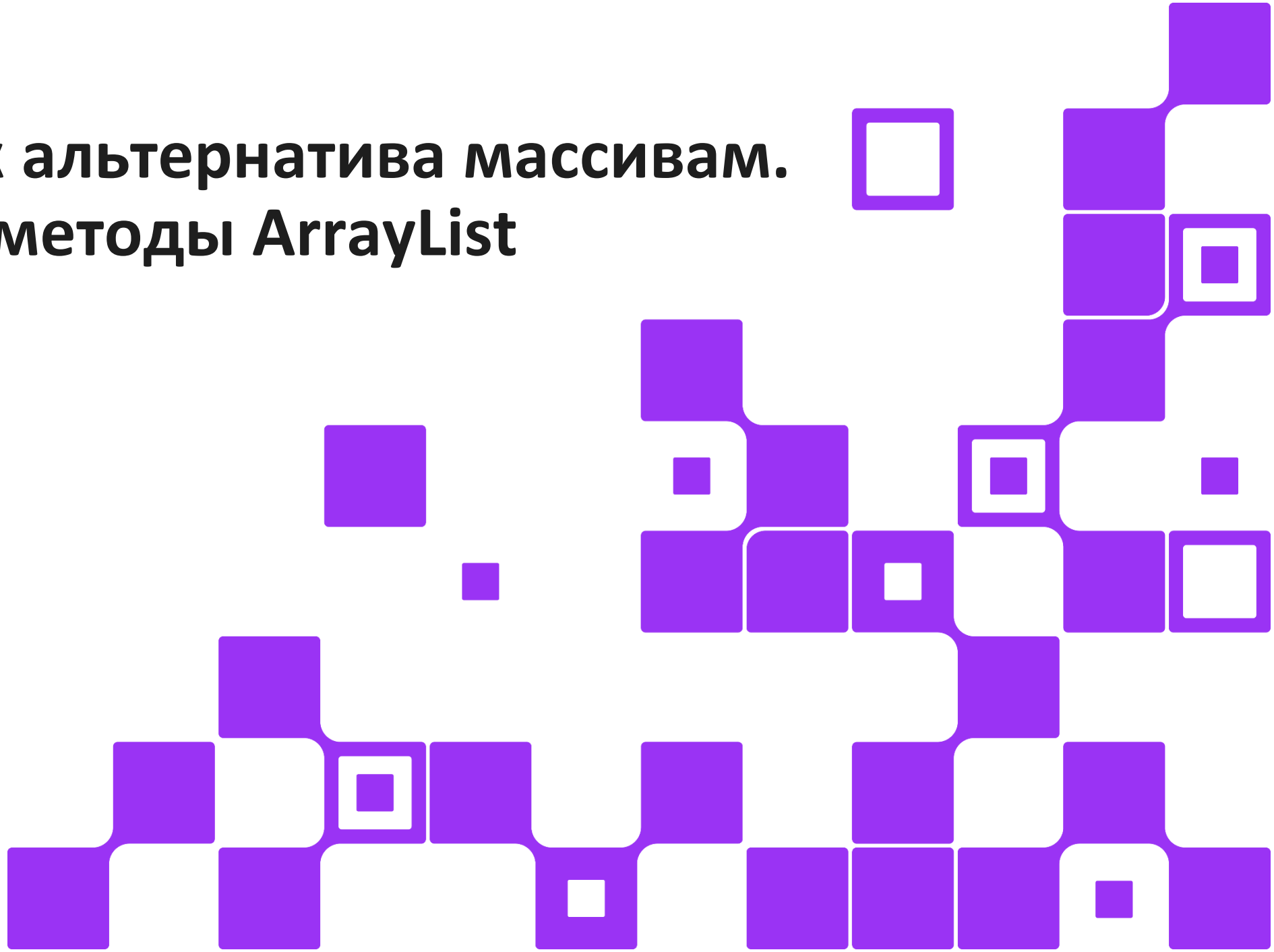
ArrayList как альтернатива массивам. Создание и методы ArrayList



IThub

IThub

Класс ArrayList и интерфейс List

class ArrayList<E> extends AbstractList<E> implements List<E>

- в E указывается тип объектов, которые будет хранить список
- ArrayList представляет простой список, аналогичный массиву, за тем исключением, что количество элементов в нем не фиксировано
- списковые массивы создаются с начальным размером. В случае превышения этого размера массив автоматически увеличивается, а когда объекты удаляются, размер массива может быть уменьшен

! эта реализация не синхронизирована (если несколько потоков одновременно обращаются к экземпляру ArrayList , и хотя бы один из них изменяет структуру списка, необходимо выполнить внешнюю синхронизацию)

Часто используемы методы класса ArrayList

Модификатор и тип	Метод	Описание
void	add(int index, E element)	Вставляет указанный элемент в указанную позицию этого списка. Ранее существовавшие элементы в точке вставки или после нее сдвигаются вправо. В итоге элементы списка не перезаписываются
boolean	addAll(int index, Collection<? extends E> c)	Вставляет все элементы указанной коллекции в этот список в указанную позицию. Возвращает true, если вызывающий список изменился, или false в противном случае
E	get(int index)	Возвращает элемент в указанной позиции этого списка
int	indexOf(Object o)	Возвращает индекс первого вхождения указанного элемента в этот список или -1, если список не содержит элемента
E	remove(int index)	Удаляет из вызывающего списка элемент в позиции index и возвращает удаленный элемент. Результирующий список уплотняется, т.е. индексы последующих элементов уменьшаются на единицу
boolean	remove(Object o)	Удаляет первое вхождение указанного элемента из этого списка, если оно присутствует
E	set(int index, E element)	Присваивает element значение по индексу, указанному в index внутри вызывающего списка. Возвращает старое значение

```java
public void arrayListDemo() {
    ArrayList<String> list = new ArrayList<>();
    list.add("Monday");
    list.add("Tuesday");
    list.add("Wednesday");
    list.add("Thursday");
    list.add("Friday");
    list.add("Saturday");
    list.add("Sunday");

    System.out.println("ArrayList elements size = " + list.size() + ". Elements:");
    list.forEach(System.out::println);

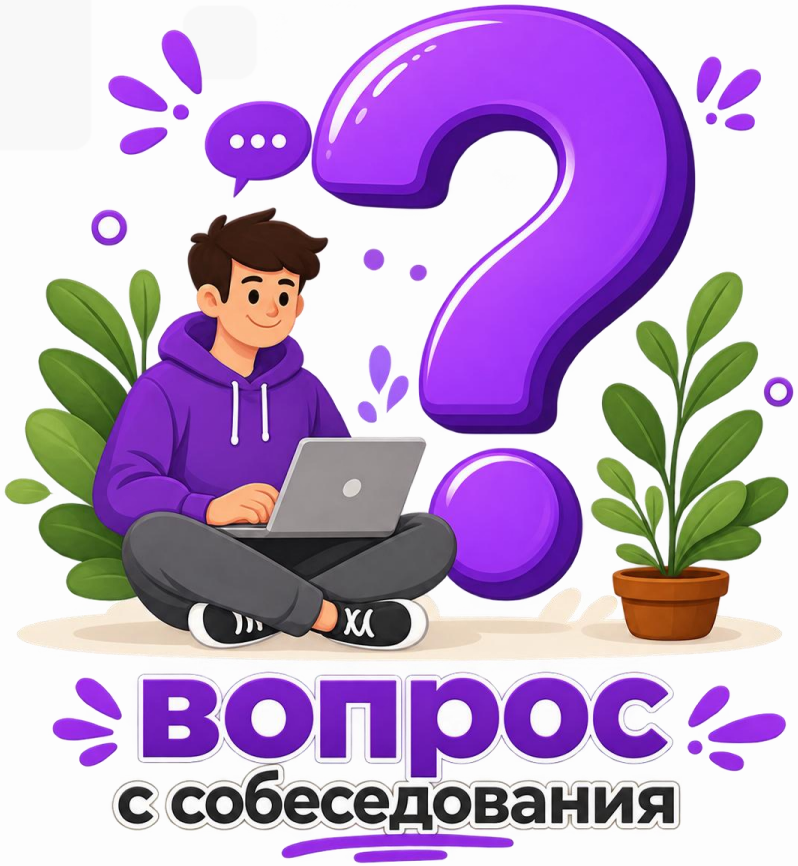
    list.remove("Monday");
    System.out.println("\nArrayList elements size after remove = " + list.size() + ". Elements:");
    list.forEach(System.out::println);
}
```

```
ArrayList elements size = 7. Elements:
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
```

```
ArrayList elements size after remove = 6. Elements:
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
```

IThub

IThub

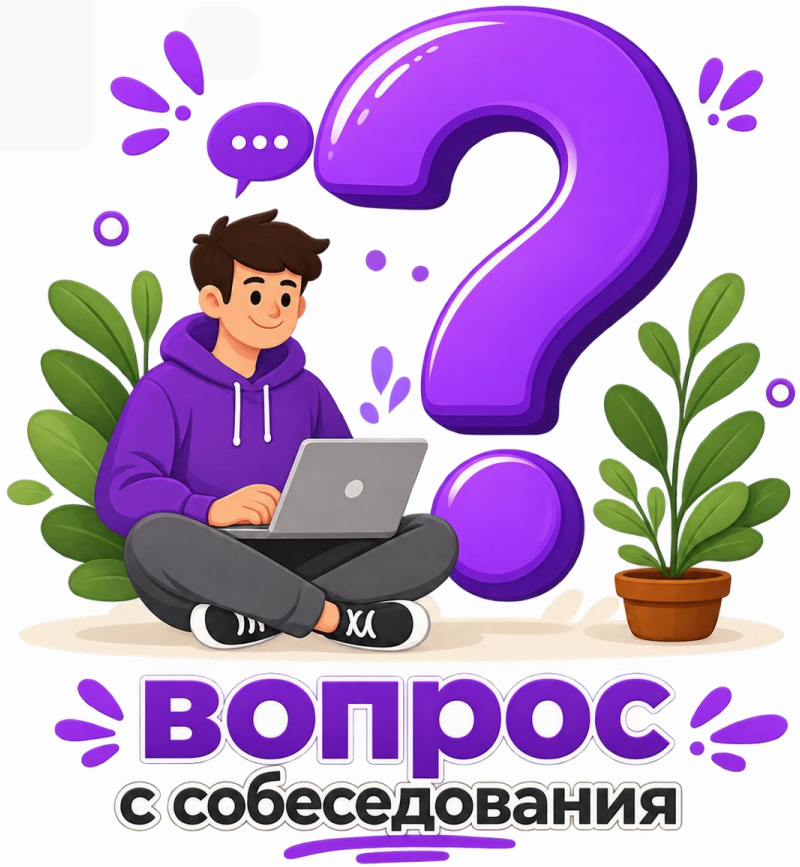


Как происходит удаление элементов из ArrayList? Как меняется в этом случае размер ArrayList?

При удалении произвольного элемента из списка, все элементы, находящиеся «правее» смещаются на одну ячейку влево и реальный размер массива (его емкость, capacity) не изменяется никак.

Механизм автоматического «расширения» массива существует, а вот автоматического «сжатия» нет, можно только явно выполнить «сжатие» командой trimToSize().

IThub



В реализации класса ArrayList есть следующие поля:
Object[ ] elementData, int size. Объясните, зачем хранить отдельно size, если всегда можно взять elementData.length?

Размер массива elementData представляет собой вместимость (capacity) ArrayList, которая всегда больше переменной size - реального количества хранимых элементов. При необходимости вместимость автоматически возрастает.

Задание

Необходимо написать программу «Электронный журнал учеников».

Для этого создать 2 базовых класса:

1. создать класс **Student**. Класс должен содержать поля: имя(String), возраст (int), список дисциплин (ArrayList<Discipline>)
2. создать класс **Discipline** - каждая дисциплина хранит: название предмета (String); список оценок (ArrayList<Integer>).

```
School
 |
 |-- ArrayList<Student>
        |
        |-- ArrayList<Discipline>
               |
               |-- ArrayList<Integer>
```

Реализовать:

1. Добавление ученика.
2. Добавление предмета ученику.
3. Добавление оценки по предмету.
4. Вывод дневника ученика.
5. Поиск ученика с максимальным средним баллом.
6. Поиск лучшего предмета класса.
7. Удаление ученика.


```
ArrayList<Integer> list = new ArrayList<>(List.of(2, 4, 5));
```